Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our privacy policy.

Manage Reject Accept

Why do good people write bad software?

Answer: Because good software requires time that you don't always have

JOS VISSER
DEC 17, 2026

3 1 1 Share

(Like this article? Read more Wednesday Wisdom! No time to read? No worries! This article is also available as a podcast). You can also ask your questions to our specially trained GPT)

> *This is an updated version of the first ever Wednesday Wisdom, which was published more than four years ago on 12/1/2021 inside an intranet far, far away…*

In my career to date, I have written terrible software and I have seen quite a few terrible systems. A lot of this software is in production with various levels of success. It is not an exaggeration to say that if we built airplanes in the same way we build software, they would be falling out of the sky on a regular basis.

When confronted with a terrible mess, I try to be open minded and **not** automatically assume that the people who designed and developed that mess were muppets who did not know what they were doing, because, in my experience, that is usually not the case. And even when it is, it is typically not helpful to broadcast that message.

> Q: Why not?
> A: Because all communication is strategic and what would be your goal in saying this?

The worst software I ever wrote in a professional capacity was the pager module for an HP OpenView installation at the Dutch state telco. I was part of a project team that sought to create a consolidated system management environment for the hundreds of HP and Sun computers that operated the phone switches and we needed to page someone when an alert had gone unacknowledged for some time. That is not necessarily a hard problem, but neither is it entirely trivial, especially given the many constraints of the systems we were working on. The task to write the pager module befell to me and I came up with a crappy design that I subsequently implemented. Months later, when some routine maintenance was needed, my colleague Shoenix (a handle, obviously) came to me and bluntly asked: "Dude, what were you on when you wrote that pager module? This is the worst piece of code I have seen in my life!"

> Video tip: Why are the Dutch so direct?

I consider myself a decent software engineer with a lot of focus on clarity of design and implementation. Why did I write this piece of bad software?

The answer of course was time pressure. I had a lot going on at the time (professionally) and had postponed thinking about the pager module until the deadline was upon me. Then, in a night-time rush of vi and cc, I hacked out some code, tested it, and put it in production. During the development and early testing, I ran into all sorts of problems that were related to the crappy (or better: lack of) design, but since I can code like a demon, I managed to solve all of these and get the software out of the door in time.

Software, like good wine, requires time, and I had not taken that time…

As a general rule, first designs are not very good, because it is quite rare to grasp all the requirements, all the edge cases, and all the details of the runtime environment before committing to a solution in code. A first design needs to age a bit, be subject to feedback, and maybe partially implemented in order to learn what the "right" solution is. It is an age-old adagium of software engineering that only after you put your first version into production, you really understand what you should have built in the first place. But, once the thing is running, who has the time to go back and do it right?

If you have the time, a good way to improve ideas is to write them down. This has two advantages. First: The act of writing ideas down forces you to be specific about what the ideas actually are, which requires you to resolve any loose ends yourself, because writing is explicit and which nerd wants to be explicitly inconsistent?

> Career tip: The ability to write well is an important career amplifier.

The written form is also more convenient for soliciting feedback, which is important because people invariably find mistakes, omissions, or come up with better ideas. It's only after few rounds of feedback that the design gets into a shape worth considering for implementation. Not going through this cycle of feedback often means you are going to build something that is not very good.

All of this takes time.

> By the way: Design docs are also a good vehicle for approval processes. However, I am of the opinion that you should never approve design docs.

Another area in which time, or the perceived lack thereof, impacts software quality is technical debt. Every piece of software is laden down with technical debt and paying down that debt should be projects in themselves and part of every other project to boot. Not doing that eventually leads to tech bankruptcy.

Unfortunately, paying down technical debt takes time. Not taking that time means that you have to start hacking around the debt, thereby *increasing* it and introducing ever more hard-to-find bugs. The result of not taking the time to address technical debt is more bad code, which makes future software development go even slower, as people have to deal with more hard-to-understand spaghetti code and hidden dependencies

> At one big tech company I worked for, we could not release a new version of one of our flagship products for months because we could not get a "green" build and any attempts to stabilize the situation were thwarted by the constant backporting of bug fixes from the release branch to the trunk that we were trying to fix.

So why do good people write bad code? Because they do not get, or do not take, enough time. Developing good software is a difficult and slow process. If you don't take the time to do it right, you will be digging yourself into a deeper and deeper hole with every PR you merge.

Does this mean that I propose never putting any time constraints on software development?

Certainly not! Software quality is but one variable in the equation that makes a successful company and there are others, like business opportunities and external deadlines. At the end of the day, quality is nothing more than fitness for use and sometimes bad software can be incredibly fit for use, like the pager module that I started this article with.

As I mentioned last week: It is often (maybe always?) more important to build the right thing than to build the thing right. We have many examples of beautiful software that is doing nothing because the company that built it went under, beaten by competitors that had a crappy but appealing solution out of the door faster. Once you "win" in your market and become big, you often have the time, but if not, certainly the money, to start a massive payback of that technical debt you incurred when you were focusing on winning the race.

Perhaps the hallmark of a good engineer is not that they always write good software, but that they can get bad software working and out of the door in time when the situation requires it.

You know what takes almost no time?
Subscribing to Wednesday Wisdom!

Subscribed

3 Likes · 1 Restack

Discussion about this video

Comments Restacks

Write a comment…

Anup Buchke 6h

The ROI of reducing technical debt is not tangible unfortunately especially when all the other promo packets are focused around…" I implemented X which gave $M gains" Vs "I cleaned up bugs/refactored code for support future scaling" . We all know who will win the promo battle(in most companies).

LIKE REPLY SHARE

Wednesday Wisdom Podcast

This is where I read my Wednesday Wisdom articles out loud in a terrible Dutch accent and without sound effects.

Subscribed

Listen on

Substack App Apple Podcasts RSS Feed

Appears in episode

Jos Visser

Recent Episodes

Misinterpreting SRE
DEC 10 · JOS VISSER

Stress!
DEC 3 · JOS VISSER

TG Special: A reader question
NOV 26 · JOS VISSER

Respect what came before
NOV 19 · JOS VISSER

Short: Are you a koala or a duck?
NOV 12 · JOS VISSER

The gift of modesty
NOV 5 · JOS VISSER

More things I learned running a small business
OCT 29 · JOS VISSER

© 2025 Josephus C.W. Visser · Privacy · Terms · Collection notice

Start your Substack Get the app

Substack is the home for great culture